

INSTITUTO DE EDUCACIÓN SECUNDARIA



Código de centro: 28074852



Cofinanciado por la Unión Europea



Práctica OpenCode UT1. Documentar y versionar con un agente

Resolución completa — enunciados, comandos y salidas reales

Alumno: RD · **Fecha:** 16 de octubre de 2026

Entorno: OpenCode v2.0.16 (modelo activo comprobado con `/models`: DeepSeek V4.1 Flash Free, proveedor opencode) · Node.js v22.22.1 · npm 11.21.1 (fijado con corepack) · Git 2.53.0 · LibreOffice 26.2.5.2 · markdown-it-py 3.0.0 · Ubuntu (GNOME)

Repositorio: `/home/rd/Practica2OpenCode` · rama `main` · 15 commits (el último, este informe) · sin remoto configurado · subidas a remoto prohibidas por configuración

Este documento resuelve los nueve ejercicios de la práctica. Para cada uno se incluye el enunciado, qué se hizo y la evidencia en forma de **comandos ejecutados y sus salidas reales** (algunas salidas largas se abrevian indicándolo). Además se incluye una captura de pantalla real del bloqueo de `git push`. El diario completo de prompts se mantiene en `prompts.md`, dentro del repositorio.

Ejercicio 1 — Primer contacto (0,5 puntos)

> **Enunciado.** Copia gestor-tareas a tu carpeta de trabajo, inicializa un repositorio Git y haz un commit con el proyecto tal cual («Proyecto de partida»). Arranca opencode dentro de la carpeta y comprueba el modelo activo con /models. En modo Plan pídele: «explica qué hace este proyecto, qué ficheros lo componen y cómo se ejecuta». Captura la respuesta y comprueba tú que los ficheros y los comandos que cita existen de verdad.

Qué se hizo

- Se copió `gestor-tareas` a la carpeta de trabajo y se inicializó el repositorio con la rama `main`; primer commit con el proyecto sin tocar: **«Proyecto de partida»**.
- Se comprobó el modelo activo con `/models`: **DeepSeek V4.1 Flash Free** (modelo gratuito, como advierte la práctica: conviene ser explícito en los prompts y verificar más).
- En **modo Plan** se pidió la explicación del proyecto. Respuesta del agente (resumen): «*gestor-tareas es una aplicación de consola escrita en JavaScript (CommonJS) sin dependencias externas que gestiona tareas persistidas en `datos/tareas.json`. La forman cuatro ficheros: `package.json` (metadatos y scripts npm `start` y `test`), `src/cli.js` (interfaz de línea de comandos: interpreta `process.argv` y despacha las órdenes `anadir`, `completar`, `eliminar`, `listar` y `resumen`; sin orden imprime la ayuda), `src/tareas.js` (lógica de negocio y persistencia en JSON; exporta seis funciones) y `datos/tareas.json` (array de tareas con `id`, `texto`, `prioridad` y `hecha`). Se ejecuta con `node src/cli.js <orden>` o `npm start -- <orden>`, y las pruebas (cuando existan) con `npm test` (runner `node:test`)*».
- **Verificación propia** (la evidencia es la salida de los comandos, no la respuesta del agente): se comprobó fichero a fichero y orden a orden que todo lo citado existe y funciona.

Comandos y salidas reales

```
$ ls -R
.:
datos
package.json
src

./datos:
tareas.json

./src:
cli.js
tareas.js

$ node --version
```

```
v22.22.1
```

```
$ node src/cli.js
Uso: node src/cli.js anadir|completar|eliminar|listar|resumen
```

```
$ node src/cli.js listar
[1] (p1) Preparar el entorno de despliegue
[3] (p2) Configurar el repositorio remoto
[2] (p3) Escribir el README del proyecto
```

```
$ node src/cli.js resumen
{ total: 3, hechas: 0, pendientes: 3 }
```

Commit inicial

```
$ git init -b main
$ git add -A
$ git commit -m "Proyecto de partida"
[main (commit-raíz) 2839e46] Proyecto de partida
 4 files changed, 91 insertions(+)
 create mode 100644 datos/tareas.json
 create mode 100644 package.json
 create mode 100644 src/cli.js
 create mode 100644 src/tareas.js
```

Observación. Ya en este primer contacto se detectó una anomalía: al probar `completar`, la tarea marcada no era la esperada. Se confirmó y corrigió más adelante (ejercicio 6), que es exactamente donde la práctica lo pide.

Ejercicio 2 — AGENTS.md (1 punto)

> **Enunciado.** Ejecuta /init para que genere el fichero AGENTS.md. Léelo entero, corrige lo que no sea cierto y añade al menos cuatro normas del proyecto: documentación y commits en español, comentarios JSDoc en src/, mensajes de commit «Añade…/Corrige…», y prohibición de subir cambios al remoto. Explica para qué sirve este fichero y por qué se versiona.

Qué se hizo

- Se generó el borrador de `AGENTS.md` (equivalente a `/init`) explorando el repositorio, y se revisó **línea a línea**.
- **Error detectado y corregido:** el borrador se inventó una carpeta `test/` que no existía. Se comprobó con `ls` y se eliminó la referencia:

```
$ ls -d test
ls: no se puede acceder a 'test': No existe el archivo o directorio
```

- También se comprobó que los comandos alternativos que cita el fichero funcionan:

```
$ npm start -- listar

> node src/cli.js listar

[1] (p1) Preparar el entorno de despliegue
[3] (p2) Configurar el repositorio remoto
[2] (p3) Escribir el README del proyecto
```

Contenido final de AGENTS.md

```
# AGENTS.md

Guía para agentes de IA que trabajen en este repositorio.

## Descripción del proyecto

`gestor-tareas` es una aplicación de consola escrita en JavaScript
(CommonJS) para gestionar tareas. No tiene dependencias externas y guarda
los datos en `datos/tareas.json`.

## Estructura

- `src/cli.js`: interfaz de línea de comandos; interpreta `process.argv`
  y despacha las órdenes `anadir`, `completar`, `eliminar`, `listar` y `resumen`.
- `src/tareas.js`: lógica de negocio y persistencia sobre el fichero JSON.
```

```
- `datos/tareas.json`: almacén de datos (array de tareas).

(No hay más carpetas: el proyecto todavía no incluye pruebas.)

## Comandos

- Ejecutar la aplicación: `node src/cli.js <orden>` o `npm start -- <orden>`.
- Ejecutar las pruebas: `npm test` (runner integrado `node:test`). Cuando se
  añadan, las pruebas vivirán en `test/`.

## Normas del proyecto

1. La documentación y los mensajes de commit se escriben en español.
2. Documentar con JSDoc todas las funciones de `src/`.
3. Mensajes de commit con el formato «Añade…» o «Corrige…».
4. Prohibido subir cambios al repositorio remoto (`git push`).
```

Para qué sirve AGENTS.md y por qué se versiona

Es el fichero donde el proyecto describe **qué es** el repositorio y **cómo se trabaja en él**: contexto (estructura, comandos) y normas (idioma, estilo de documentación, formato de commits, prohibiciones). Los agentes de IA lo leen al empezar y así no reinventan el contexto ni proponen cambios que van contra las reglas del equipo. Se **versiona** porque es un artefacto más del proyecto: evoluciona con él, sus cambios se revisan en los diffs como cualquier otro fichero y garantiza que cualquier persona o agente que se incorpore vea las mismas normas vigentes en cada momento. En este repositorio se puede comprobar con `git log AGENTS.md`.

Ejercicio 3 — Permisos (1 punto)

> **Enunciado.** Crea el fichero opencode.json del proyecto para que el agente: pregunte antes de ejecutar órdenes del sistema, tenga prohibido git push y rm, no pueda leer .env ni escribir fuera de src/, test/ y docs/, y no pueda compartir la sesión. Demuestra con una captura que un git push pedido al agente queda bloqueado.

Fichero opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "share": "disabled",
  "permissions": [
    { "action": "shell", "resource": "*", "effect": "ask" },
    { "action": "shell", "resource": "git push *", "effect": "deny" },
    { "action": "shell", "resource": "rm *", "effect": "deny" },
    { "action": "read", "resource": "*.env*", "effect": "deny" },
    { "action": "edit", "resource": "*", "effect": "deny" },
    { "action": "edit", "resource": "src/*", "effect": "allow" },
    { "action": "edit", "resource": "test/*", "effect": "allow" },
    { "action": "edit", "resource": "docs/*", "effect": "allow" },
    { "action": "edit", "resource": "README.md", "effect": "allow" },
    { "action": "edit", "resource": "AGENTS.md", "effect": "allow" },
    { "action": "edit", "resource": "prompts.md", "effect": "allow" },
    { "action": "edit", "resource": "opencode.json", "effect": "allow" }
  ]
}
```

Las reglas se evalúan en orden y **gana la última que coincide**; por eso las prohibiciones y los permisos concretos van después de las reglas generales:

Regla	Efecto
`shell:* → ask`	**Pregunta** antes de ejecutar cualquier orden del sistema (con diálogo de aprobación).
`git push * → deny`	Subida al remoto **prohibida** (ni siquiera se ejecuta).
`rm * → deny`	Borrado con `rm` **prohibido**.
`read *.env* → deny`	No puede leer ficheros `.env` (secretos).
`edit * → deny`	No puede escribir nada salvo lo que permiten las reglas siguientes.
`edit src/*, test/*, docs/* → allow`	Escritura permitida solo en código, pruebas y documentación.

`edit README.md, AGENTS.md, prompts.md, opencode.json → allow`	Excepciones para los ficheros de la raíz que la propia práctica exige crear y editar (ver anexo B).
`"share": "disabled"`	La sesión no se puede compartir (campo soportado por compatibilidad; `/share` existe en OpenCode V2 y respeta esta configuración).

Demostración: git push bloqueado

```
$ git push origin main
PERMISO DENEGADO: Permission denied: shell
(la orden no llega a ejecutarse: la política de permisos la deniega antes)
```



Captura real de la sesión: el intento de `git push origin main` pedido al agente queda bloqueado («Permission denied: shell»).

Demostración adicional: escritura fuera de src/, test/ y docs/

```
(herramienta de escritura → fichero notas.txt en la raíz)
PERMISO DENEGADO: Permission denied: edit
```

Comprobaciones de apoyo

```
$ git remote -v
(sin salida: no hay remoto configurado)
```

```
$ git branch -vv
* main e878340 Añade registro de prompts y balance de la práctica
```

Además de la política de permisos, no existe remoto configurado, de modo que un `git push` tampoco tendría destino: doble protección.

Ejercicio 4 — Documentación del código (1,5 puntos)

> **Enunciado.** Pide al agente que documente con JSDoc las seis funciones de src/tareas.js (@param, @returns, @throws y un @example). Revisa una por una: la descripción debe coincidir con lo que hace el código. Anota al menos una descripción incorrecta o inventada y corrígela.

Qué se hizo

- Se documentaron con JSDoc las seis funciones exportadas (`leer`, `anadir`, `completar`, `eliminar`, `pendientes`, `resumen`) y también la interna `guardar`, con descripción, `@param` (cuando procede), `@returns`, `@throws` y `@example`.
- Se revisaron una por una comparando la documentación con el código real.
- **Incidencias anotadas y corregidas:**
  - **Descripción incorrecta** en `resumen()`: decía que devolvía un número de tareas pendientes, cuando devuelve un objeto. Corregida antes del commit (antes/después más abajo).
  - **Incoherencia código–documentación** en `completar()`: la descripción habla del *id* de la tarea, pero el código usaba el argumento como *índice* del array. Se dejó anotada y se resolvió con la prueba y la corrección del ejercicio 6 (donde el JSDoc se actualizó para reflejar el error que se lanza).

JSDoc final de las seis funciones

```
/**
 * Lee todas las tareas almacenadas.
 *
 * Si el fichero todavía no existe, devuelve un array vacío.
 *
 * @returns {Array<{id: number, texto: string, prioridad: number, hecha: boolean}>}
 * Lista completa de tareas tal y como están guardadas.
 * @throws {SyntaxError} Si el fichero existe pero su contenido no es JSON válido.
 * @example
 * const tareas = leer();
 * console.log(`Hay ${tareas.length} tareas`);
 */
function leer() { ... }

/**
 * Guarda la lista completa de tareas, sustituyendo el contenido anterior.
 *
 * Crea la carpeta del fichero si no existiera.
 *
 * @param {Array<{id: number, texto: string, prioridad: number, hecha: boolean}>} tareas
 * Lista completa de tareas que se quiere persistir.
```

```
 * @returns {void}
 * @throws {Error} Si no se puede escribir el fichero (por ejemplo, por permisos).
 * @example
 * guardar([{ id: 1, texto: 'Comprar pan', prioridad: 2, hecha: false }]);
 */
function guardar(tareas) { ... }

/**
 * Añade una tarea nueva con un identificador consecutivo.
 *
 * El identificador asignado es el mayor de los existentes más uno
 * (la primera tarea recibe el id 1).
 *
 * @param {string} texto Descripción de la tarea.
 * @param {number} [prioridad=2] Prioridad de la tarea; 1 es la más alta.
 * @returns {number} Identificador asignado a la tarea creada.
 * @throws {Error} Si no se puede escribir el fichero de datos.
 * @example
 * const id = anadir('Escribir el README');
 * console.log(id); // 4
 */
function anadir(texto, prioridad = 2) { ... }

/**
 * Marca como hecha la tarea indicada.
 *
 * @param {number} id Identificador de la tarea que se quiere completar.
 * @returns {{id: number, texto: string, prioridad: number, hecha: boolean}}
 * La tarea actualizada.
 * @throws {Error} Si no existe ninguna tarea con el id indicado.
 * @example
 * completar(1); // la tarea con id 1 pasa a estar hecha
 */
function completar(id) { ... }

/**
 * Elimina la tarea indicada.
 *
 * @param {number} id Identificador de la tarea que se quiere borrar.
 * @returns {number} Número de tareas que quedan tras el borrado.
 * @throws {Error} Si no se puede escribir el fichero de datos.
 * @example
 * const quedan = eliminar(2);
 * console.log(quedan); // 2
```

```
 */
function eliminar(id) { ... }

/**
 * Devuelve las tareas pendientes ordenadas por prioridad.
 *
 * La tarea con prioridad 1 aparece primero.
 *
 * @returns {Array<{id: number, texto: string, prioridad: number, hecha: boolean}>}
 * Tareas con `hecha === false`, de menor a mayor prioridad.
 * @throws {SyntaxError} Si el fichero de datos no es JSON válido.
 * @example
 * pendientes().forEach((t) => console.log(t.texto));
 */
function pendientes() { ... }

/**
 * Calcula un resumen del estado de las tareas.
 *
 * @returns {{total: number, hechas: number, pendientes: number}}
 * Objeto con el número total de tareas, las ya completadas y las pendientes.
 * @throws {SyntaxError} Si el fichero de datos no es JSON válido.
 * @example
 * resumen(); // { total: 3, hechas: 0, pendientes: 3 }
 */
function resumen() { ... }
```

Corrección de la descripción incorrecta (resumen)

Antes (incorrecta):

```
/**
 * Cuenta las tareas pendientes.
 *
 * @returns {number} Número de tareas que aún no están hechas.
 * @throws {SyntaxError} Si el fichero de datos no es JSON válido.
 * @example
 * resumen(); // 3
 */
```

Después (corregida, comprobada contra la salida real `{ total: 3, hechas: 0, pendientes: 3 }`):

```
/**
 * Calcula un resumen del estado de las tareas.
 *
```

```
 * @returns {{total: number, hechas: number, pendientes: number}}
 * Objeto con el número total de tareas, las ya completadas y las pendientes.
 * @throws {SyntaxError} Si el fichero de datos no es JSON válido.
 * @example
 * resumen(); // { total: 3, hechas: 0, pendientes: 3 }
 */
```

Función	Revisión (¿coincide con el código?)	Resultado
`leer()`	Devuelve `[]` si no hay fichero; `JSON.parse` puede lanzar `SyntaxError`.	**✔ correcta**
`guardar()`	Sustituye el contenido y crea la carpeta si falta.	**✔ correcta**
`anadir()`	Id = máximo + 1; prioridad por defecto 2; la tarea nace sin hecha.	**✔ correcta**
`completar()`	La descripción habla del *id*; el código usaba el argumento como índice.	**✘ incoherencia → ejercicio 6**
`eliminar()`	Filtra por `id` y devuelve cuántas quedan.	**✔ correcta**
`resumen()`	Decía devolver un número; devuelve `{total, hechas, pendientes}`.	**✘ corregida aquí**

Ejercicio 5 — README (1,5 puntos)

> **Enunciado.** Pídele un README.md con: título, descripción, requisitos, instalación, uso con un ejemplo por cada orden de la CLI, estructura de carpetas, formato de datos/tareas.json, autoría y licencia. Ejecuta todos los comandos del README: corrige los que no funcionen o den una salida distinta a la documentada y deja constancia de la corrección.

Qué se hizo

- Se escribió un README con **todas** las secciones pedidas: título, descripción, requisitos, instalación, uso con un ejemplo de cada orden (`anadir`, `listar`, `completar`, `eliminar`, `resumen`), atajos con npm, estructura de carpetas, formato de `datos/tareas.json`, autoría y licencia.
- Se **ejecutaron todos los comandos** del README, restaurando el estado de los datos antes de cada ejemplo para compararlo con lo documentado (el fichero de datos está en Git, así que se puede volver atrás con `git restore`).
- Se corrigieron las discrepancias encontradas y quedó constancia en `prompts.md` y en el commit correspondiente.

Ejecución completa de verificación (salida real)

```
=== [1] node src/cli.js (ayuda) ===
Uso: node src/cli.js anadir|completar|eliminar|listar|resumen
=== [2] listar (documentado: 3 tareas) ===
[1] (p1) Preparar el entorno de despliegue
[3] (p2) Configurar el repositorio remoto
[2] (p3) Escribir el README del proyecto
=== [3] anadir (documentado: 'Tarea 4 creada con prioridad 2') ===
Tarea creada con id 4
=== [4] eliminar 4 (documentado: 'Eliminada; quedan 3 tareas') ===
Eliminada
=== [5] completar 1 (documentado: tarea id 1) ===
{
  id: 2,
  texto: 'Escribir el README del proyecto',
  prioridad: 3,
  hecha: true
}
=== [6] resumen (documentado: { total: 3, hechas: 0, pendientes: 3 }) ===
{ total: 3, hechas: 0, pendientes: 3 }
=== [7] npm run listar (documentado como atajo) ===
npm error Missing script: "listar"
npm error
npm error Did you mean one of these?
```

```
npm error npm star # Mark your favorite packages
npm error npm unstar # Remove an item from your favorite packages
npm error
npm error To see a list of scripts, run:
npm error npm run
npm error A complete log of this run can be found in: /home/rd/.npm/_logs/2026-10-16T09_03_10_286Z-debug-0.log
exit=1
```

Discrepancias encontradas y correcciones

Elemento	Documentado	Real	Corrección
`anadir`	`Tarea 4 creada con prioridad 2`	`Tarea creada con id 4`	Salida corregida en el README; además se aclaró que la CLI solo acepta el texto (la prioridad no se puede indicar todavía).
`eliminar`	`Eliminada; quedan 3 tareas`	`Eliminada`	Salida corregida y se anotó que si el id no existe los datos no cambian.
Atajo npm	`npm run listar`	`npm error Missing script: "listar"`	Atajo inexistente eliminado; se conserva el correcto `npm start -- listar`.
`completar`	Muestra la tarea con id 1	Mostraba la tarea con id 2	**No era un error de documentación**: era un fallo del código (el README documentaba el comportamiento correcto). Se resolvió en el ejercicio 6.

Commit de la corrección:

```
$ git log --oneline -1 478b9c2
478b9c2 Corrige el README tras ejecutar todos los ejemplos
```

Ejercicio 6 — El error (2 puntos)

> **Enunciado.** El proyecto tiene un fallo: node src/cli.js completar 1 no marca la tarea que debería. Sin decirle cuál es el fallo, pide al agente que escriba en test/ una prueba con node:test que lo demuestre y ejecútala con npm test (debe fallar). Solo después, pídele que corrija src/tareas.js y vuelve a ejecutar las pruebas (deben pasar). Adjunta las dos salidas y explica en qué consistía el error.

La prueba (test/tareas.test.js)

```
/**
 * @file Pruebas de `src/tareas.js` con el runner integrado de Node
 * (`node:test`). Se ejecutan con `npm test`.
 */

const { test, beforeEach, afterEach } = require('node:test');
const assert = require('node:assert/strict');
const fs = require('node:fs');
const path = require('node:path');

const tareas = require('../src/tareas');

const FICHERO = path.join(__dirname, '..', 'datos', 'tareas.json');

// Copia de seguridad del fichero de datos para restaurarlo tras cada prueba.
let copia;

beforeEach(() => {
  copia = fs.readFileSync(FICHERO, 'utf8');
});

afterEach(() => {
  fs.writeFileSync(FICHERO, copia);
});

test('completar(1) marca como hecha la tarea con id 1 (y solo esa)', () => {
  tareas.completar(1);

  const despues = tareas.leer();
  const tarea1 = despues.find((t) => t.id === 1);
  const tarea2 = despues.find((t) => t.id === 2);
  const tarea3 = despues.find((t) => t.id === 3);

  assert.equal(tarea1.hecha, true, 'la tarea con id 1 debería estar hecha');
```

```
  assert.equal(tarea2.hecha, false, 'la tarea con id 2 no debería tocarse');
  assert.equal(tarea3.hecha, false, 'la tarea con id 3 no debería tocarse');
});
```

Salida 1: las pruebas fallan (salida abreviada)

```
$ npm test

> gestor-tareas@0.1.0 test
> node --test

TAP version 13
# Subtest: completar(1) marca como hecha la tarea con id 1 (y solo esa)
not ok 1 - completar(1) marca como hecha la tarea con id 1 (y solo esa)
  ---
  duration_ms: 8.487181
  type: 'test'
  location: '/home/rd/Practica2OpenCode/test/tareas.test.js:26:1'
  failureType: 'testCodeFailure'
  error: |-
    la tarea con id 1 debería estar hecha

    false !== true

  code: 'ERR_ASSERTION'
  name: 'AssertionError'
  expected: true
  actual: false
  operator: 'strictEqual'
  ...
1..1
# tests 1
# suites 0
# pass 0
# fail 1
# cancelled 0
# skipped 0
# todo 0
```

En qué consistía el error

La función `completar(id)` usaba el argumento como **índice del array**:

```
function completar(id) {
  const tareas = leer();
```

```
  tareas[id].hecha = true; // ← ¡id como índice!
  guardar(tareas);
  return tareas[id];
}
```

Como los `id` de las tareas empiezan en 1 y los índices del array en 0, había un desfase de uno: `completar 1` marcaba la **segunda** tarea, `completar 2` la tercera y `completar 3` provocaba un `TypeError` (índice fuera del array). El error pasaba desapercibido porque «parecía funcionar»: cambiaba algo y no lanzaba error en el caso 1.

Corrección (diff revisado antes de confirmar)

```
$ git diff --staged
diff --git a/src/tareas.js b/src/tareas.js
index 0830576..2147eac 100644
--- a/src/tareas.js
+++ b/src/tareas.js
@@ -74,15 +74,17 @@ function anadir(texto, prioridad = 2) {
  * @param {number} id Identificador de la tarea que se quiere completar.
  * @returns {{id: number, texto: string, prioridad: number, hecha: boolean}}
  * La tarea actualizada.
- * @throws {TypeError} Si no existe ninguna tarea con ese identificador.
+ * @throws {Error} Si no existe ninguna tarea con el id indicado.
  * @example
  * completar(1); // la tarea con id 1 pasa a estar hecha
  */
 function completar(id) {
   const tareas = leer();
- tareas[id].hecha = true;
+ const tarea = tareas.find((t) => t.id === id);
+ if (!tarea) throw new Error(`No existe ninguna tarea con el id ${id}`);
+ tarea.hecha = true;
   guardar(tareas);
- return tareas[id];
+ return tarea;
 }

 /**
```

La corrección busca la tarea por su `id` con `find()` y lanza un error claro si no existe. Además se actualizó el `@throws` del JSDoc para reflejarlo.

Salida 2: las pruebas pasan (salida abreviada)

```
$ npm test
```

```
> gestor-tareas@0.1.0 test
> node --test

TAP version 13
# Subtest: completar(1) marca como hecha la tarea con id 1 (y solo esa)
ok 1 - completar(1) marca como hecha la tarea con id 1 (y solo esa)
  ---
  duration_ms: 4.466752
  type: 'test'
  ...
1..1
# tests 1
# suites 0
# pass 1
# fail 0
# cancelled 0
# skipped 0
# todo 0
```

Comprobación final con el comando real

```
$ node src/cli.js completar 1
{
  id: 1,
  texto: 'Preparar el entorno de despliegue',
  prioridad: 1,
  hecha: true
}
```

La salida coincide ya con la documentada en el README. La prueba protege los datos reales: antes de cada test guarda una copia de `datos/tareas.json` y la restaura al terminar.

Ejercicio 7 — Git con supervisión (1,5 puntos)

> **Enunciado.** Pide al agente que prepare los cambios en commits separados (documentación, pruebas y corrección) con mensajes en español. Antes de cada commit revisa git diff --staged. Después, crea con él docs/control-de-versiones.md: instalación y configuración de Git usada, flujo de trabajo, convención de mensajes y cómo revisar lo que hace un agente antes de confirmarlo. Verifica el historial con git log --oneline --graph.

Instalación y configuración usada

```
$ git --version
git version 2.53.0

$ git config --global user.name
RD

$ git config --global user.email
rd@example.com
```

Commits separados por tema

La documentación, la prueba y la corrección se prepararon en tres commits distintos, con el diff de cada uno revisado con `git diff --staged` antes de confirmarlos (en el ejercicio 6 se muestra uno de esos diffs):

```
47c1361 Añade documentación JSDoc a las funciones de src/tareas.js
852f7f8 Añade prueba de completar() con node:test
1d51aea Corrige completar() para localizar la tarea por su id
```

docs/control-de-versiones.md (resumen del contenido)

- **Instalación y configuración:** Git 2.53.0; identidad `RD <rd@example.com>`; repositorio creado con `git init -b main`; sin remoto configurado (las subidas están prohibidas por `AGENTS.md` y por `opencode.json`).
- **Flujo de trabajo:** un cambio = un tema = un commit; revisar el diff y ejecutar las pruebas antes de confirmar; mensajes en español; trabajo lineal sobre `main`; `git restore` para deshacer cambios sin confirmar.
- **Convención de mensajes:** «Añade…» para lo nuevo y «Corrige…» para los arreglos (con la excepción del commit inicial, «Proyecto de partida», fijado por el enunciado).
- **Cómo revisar lo que hace un agente antes de confirmarlo:**
    1. `git status` y `git diff`: ver qué ficheros toca y qué cambia.
    2. `git diff --staged`: revisión final de lo que se va a confirmar.
    3. Ejecutar la verificación real (`npm test` y los comandos del README): la evidencia es la salida, no la respuesta del agente.

4. Comprobar que el mensaje de commit describe de verdad el cambio.
5. Apoyarse en los permisos de `opencode.json`: aprobación de órdenes, prohibiciones explícitas y límites de escritura.
6. Consultar el historial: `git log --oneline --graph` y `git show`.

Verificación del historial

```
$ git log --oneline --graph
* e878340 Añade registro de prompts y balance de la práctica
* 4c8a986 Añade conversión del README a HTML y PDF y tabla comparativa
* f5270e7 Añade guía de control de versiones
* 01b26aa Añade la sección de pruebas al README
* 1d51aea Corrige completar() para localizar la tarea por su id
* 852f7f8 Añade prueba de completar() con node:test
* 478b9c2 Corrige el README tras ejecutar todos los ejemplos
* 69a0ca5 Añade README con la documentación del proyecto (borrador)
* 47c1361 Añade documentación JSDoc a las funciones de src/tareas.js
* 0cb78f7 Añade fichero de bloqueo generado por npm
* c45bbf3 Añade configuración de permisos de OpenCode
* 80a9b5d Añade AGENTS.md con las normas del proyecto
* d1cb56e Añade versión de npm fijada con corepack
* 2839e46 Proyecto de partida
```

> El historial anterior es el estado justo antes de crear este informe; el commit que añade el informe y su registro queda como último commit del repositorio.

Ejercicio 8 — Comparación de formatos (0,5 puntos)

> **Enunciado.** Convierte el README.md a HTML y a PDF (extensión Markdown PDF de VS Code, Pandoc o el propio navegador) y añade al README una tabla que compare los tres formatos: ventajas, inconvenientes y cuándo usar cada uno.

Qué se hizo

- Conversión reproducible y versionada: `bash docs/convertir-readme.sh` usa `docs/md-a-html.py` (Markdown → HTML con `markdown-it-py`) y LibreOffice (`soffice --headless`, HTML → PDF). El resultado vive en `docs/README.html` y `docs/README.pdf`.
- Se añadió al README la tabla comparativa de los tres formatos (incluida abajo) y se regeneraron HTML y PDF para que la contuvieran.

```
$ bash docs/convertir-readme.sh
Generado /home/rd/Practica2OpenCode/docs/README.html
convert /home/rd/Practica2OpenCode/docs/README.html as a Writer/Web document ->
/home/rd/Practica2OpenCode/docs/README.pdf using filter : writer_web_pdf_Export
Conversión completada: docs/README.html y docs/README.pdf
```

```
$ pdfinfo docs/README.pdf | grep -E "^(Pages|Title)"
Title: README
Pages: 5
```

gestor-tareas

Gestor de tareas por línea de comandos: crea, completa, elimina, lista y resume tareas, guardándolas en un fichero JSON local.

Descripción

`gestor-tareas` es un pequeño gestor de tareas escrito en JavaScript (CommonJS), sin dependencias externas. Todas las órdenes leen y escriben el fichero `datos/tareas.json`, de modo que los datos sobreviven entre ejecuciones.

Requisitos

- **Node.js 18 o superior** (probado con v22.22.1).
- **npm** (recomendado, para `npm start` y `npm test`). El proyecto fija la versión `npm@11.21.1` en el campo `packageManager` de `package.json` (compatible con `corepack`).

Instalación

No hay dependencias que instalar; basta con disponer de la carpeta del proyecto:

```
git clone <url-del-repositorio>
cd gestor-tareas
```

`npm install` es innecesario (no hay dependencias externas).

Uso

Sintaxis general:

```
node src/cli.js <orden> [argumentos]
```

Sin orden, la CLI muestra la ayuda:

```
$ node src/cli.js
Uso: node src/cli.js anadir|completar|eliminar|listar|resumen
```

Primera página de `docs/README.pdf`, generado automáticamente desde el README en Markdown.

Tabla comparativa de formatos (añadida al README)

Formato	Ventajas	Inconvenientes	Cuándo usarlo
Markdown (`README.md`)	Fácil de escribir y editar; se versiona y revisa muy bien en Git (diffs legibles); conversión sencilla a otros formatos.	No controla el diseño final; la apariencia depende del visor.	Documentación viva del repositorio: es la fuente de la que se generan los demás formatos.
HTML (`docs/README.html`)	Presentación controlada con CSS; se abre en cualquier navegador; se puede publicar en la web.	Más verboso; mezcla contenido y presentación; diffs peores de revisar.	Publicar la documentación en una web o compartirla con estilo.
PDF (`docs/README.pdf`)	Presentación fija e idéntica en cualquier equipo; cómodo de entregar e imprimir.	Formato binario: no se revisa bien en Git y no es editable con facilidad.	Entregas, impresión y documentación «congelada».

Los formatos HTML y PDF son **generados**: se regeneran desde el Markdown para que no queden desactualizados.

Ejercicio 9 — Balance (0,5 puntos)

> **Enunciado.** Responde en prompts.md: ¿cuántas correcciones tuviste que hacerle? ¿qué parte te ahorró tiempo de verdad? ¿qué habrías entregado mal si no lo hubieras comprobado?

Registro de prompts (mantenido durante la práctica)

Nº	Prompt (resumido)	Qué hizo el agente	Qué corregí yo
1	«Explica qué hace este proyecto, qué ficheros lo componen y cómo se ejecuta» (modo Plan).	Resumen correcto: describió los 4 ficheros, las órdenes de la CLI y advirtió del fallo de `completar`.	Nada; comprobé con `ls -R`, `node --version` y ejecutando la CLI que todo lo citado existe de verdad.
2	«Genera el fichero AGENTS.md del proyecto» (equivalente a `/init`).	Borrador completo con descripción, estructura, comandos y normas.	**Se inventó una carpeta `test/`** que no existía (comprobado con `ls`) y la eliminé del documento.
3	«Configura opencode.json: pide permiso para las órdenes del sistema, prohíbe `git push` y `rm`, bloquea `.env`, limita la escritura a `src/`, `test/` y `docs/`, e impide compartir la sesión».	Escribió la configuración en formato V2 (`permissions` ordenadas, `share: "disabled"`). Verificado: `git push` y una escritura a `notas.txt` quedaron denegados.	Añadí excepciones explícitas para los ficheros de la raíz que exige la propia práctica (`README.md`, `AGENTS.md`, `prompts.md`, `opencode.json`).
4	«Documenta con JSDoc las seis funciones de `src/tareas.js` con `@param`, `@returns`, `@throws` y un `@example`».	Añadió JSDoc completo a las seis funciones exportadas (y a `guardar`, interna).	**`resumen()` estaba mal documentada** (decía que devolvía un número y devuelve un objeto): la corregí. También anoté que la descripción de `completar()` no coincidía con el código (id ≠ índice), pendiente para el ejercicio 6.
5	«Escribe el README del proyecto con todas las secciones pedidas, con un ejemplo por cada orden de la CLI».	README completo y bien estructurado.	Ejecuté **todos** los comandos: **dos salidas mal documentadas** de `anadir` y `eliminar`, y un atajo inexistente (`npm run listar`) que eliminé. La discrepancia de `completar` no era de documentación: era el fallo del código.
6	«El proyecto tiene un fallo: `node src/cli.js completar 1` no marca la tarea que debería; escribe en `test/` una	Prueba con `beforeEach`/`afterEach` que protege los datos reales; `npm`	Nada; revisé que la prueba comprueba lo que dice y no toca

	prueba con `node:test` que lo demuestre» (sin decir la causa).	`test` falló mostrando el `AssertionError`.	los datos.
7	«Corrige `src/tareas.js` para que la prueba pase».	Localizó la tarea con `find` por `id` y lanza un error claro si no existe; `npm test` en verde.	Nada; revisé el diff (`@throws` actualizado incluido).
8	«Prepara los cambios en commits separados (documentación, pruebas y corrección)».	Tres commits en español siguiendo la convención «Añade…/Corrige…».	Revisé `git diff --staged` antes de cada commit.
9	«Crea `docs/control-de-versiones.md` con la instalación y configuración de Git, el flujo de trabajo, la convención de mensajes y cómo revisar el trabajo de un agente».	Documento completo y verificado con `git log --oneline --graph`.	Nada.
10	«Convierte el README a HTML y PDF y añade una tabla comparativa de los tres formatos».	Script de conversión reproducible, HTML y PDF generados, tabla añadida al README.	Nada; verifiqué el PDF visualmente (páginas, estilos y tabla).
11	«Genera el informe final en PDF con el enunciado, las soluciones y las salidas reales de cada ejercicio».	Compuso este informe en HTML (cabecera del IES, tablas y bloques de comandos) y lo convirtió a PDF con LibreOffice.	Nada; revisé el PDF página a página y ajusté el formato para que tablas y bloques de código se lean bien.

Balance

¿Cuántas correcciones tuve que hacerle? Seis correcciones relevantes: una carpeta `test/` inventada en `AGENTS.md`; una descripción JSDoc incorrecta (`resumen()`); dos salidas mal documentadas en el README (`anadir` y `eliminar`); un comando inexistente (`npm run listar`); y la detección del fallo de `completar()` (esta última no era de documentación, sino del propio código).

¿Qué parte me ahorró tiempo de verdad? Pasar del folio en blanco a un borrador revisable: `AGENTS.md`, el JSDoc de las seis funciones, el README completo y la guía de control de versiones salieron de una vez y solo hubo que validar y corregir detalles. También el trabajo mecánico: la prueba inicial, los mensajes de commit, el script de conversión Markdown → HTML → PDF y este informe.

¿Qué habría entregado mal si no lo hubiera comprobado? Una carpeta que no existe en `AGENTS.md`; una descripción falsa en el JSDoc de `resumen()`; un README con salidas que no coinciden con la realidad y un atajo que no funciona; y, lo más importante, habría dado por bueno un `completar()` que marca la tarea equivocada, porque el código «parecía» correcto hasta ejecutar la prueba. Sin la verificación (ejecutar, comparar salidas y revisar diffs) la entrega sería

incorrecta aunque lo pareciera.

Anexo A — Entorno y reproducibilidad

```
$ opencode --version
opencode v2.0.16

$ node --version
v22.22.1

$ npm --version
11.21.1 # fijado con corepack (campo packageManager de package.json)

$ git --version
git version 2.53.0
```

- Ejecutar la aplicación: `node src/cli.js <orden>` o `npm start -- <orden>`.
- Ejecutar las pruebas: `npm test` (runner integrado `node:test`).
- Regenerar los formatos del README: `bash docs/convertir-readme.sh`.
- Historial del repositorio: `git log --oneline --graph`.

Anexo B — Notas de interpretación

1. **Conversión de formatos (ejercicio 8).** La práctica propone la extensión Markdown PDF de VS Code, Pandoc o el navegador. En este entorno se usó el equivalente `markdown-it-py` + LibreOffice (Markdown → HTML → PDF), con un script versionado en `docs/` para que la conversión sea reproducible y no quede desactualizada.
2. **Límite de escritura (ejercicio 3).** «No escribir fuera de `src/`, `test/` y `docs/`» se implementó con `edit:* → deny` más excepciones explícitas para cuatro ficheros de la raíz que la **propia práctica obliga** a crear y editar: `README.md`, `AGENTS.md`, `prompts.md` y `opencode.json`. Cualquier otra ruta (incluida la raíz en general) queda denegada.
3. **share: "disabled".** Es un campo de la configuración clásica de OpenCode que la versión V2 mantiene por compatibilidad; `/share` sigue existiendo y respeta esta configuración. Se verificó que el fichero de configuración se carga sin errores (`opencode debug config`).
4. **Alcance de los permisos.** Las reglas actúan sobre las herramientas del agente (`shell`, `edit`, `read`…). Las órdenes de shell aprobadas se ejecutan con los permisos del usuario del sistema; por eso la política combina aprobación (`ask`) con prohibiciones explícitas (`deny`) para `git push`, `rm` y los secretos.